

OS Study Notes - Chapter 2

Instructions: Language of the Computer | English Version with J-format

Chapter 2 Main Idea

Computers have their own language.

That language is made of instructions.

These instructions are usually very simple.

Examples of MIPS assembly instructions:

```
add
sub
lw
sw
beq
j
```

The set of instructions that a CPU understands is called an instruction set.

This chapter uses MIPS to show how high-level programs become simple hardware instructions.

2.2 Operations of the Computer Hardware

The CPU does not usually finish a complex expression in one giant step.

It breaks complex work into simple instructions.

```
a = b + c;
```

In MIPS-style thinking:

```
add $s0, $s1, $s2
```

Meaning:

```
$s0 = $s1 + $s2
```

If the expression is bigger:

```
A = B + C + D + E + F + G
```

The CPU may receive a sequence like this:

```
add A, B, C
add A, A, D
add A, A, E
add A, A, F
add A, A, G
```

A single line in C or Java is not always one CPU instruction.

A good hardware design idea: doing many simple things well is usually cheaper than doing one complicated thing directly.

Example

```
f = (g + h) - (a + b)
```

A possible MIPS version:

```
add $t0, $s1, $s2    # $t0 = g + h
add $t1, $s3, $s4    # $t1 = a + b
sub $s0, $t0, $t1    # f = $t0 - $t1
```

\$t0 and \$t1 are temporary registers.

Even complicated programs are eventually broken down into simple instructions for the CPU.

2.3 Operands of the Computer Hardware

Instructions need operands.

When the CPU needs data, it usually reads from registers first.

A register is a very fast storage unit inside the CPU.

In 32-bit MIPS, one register holds 32 bits, which is 4 bytes.

In modern 64-bit machines, many registers hold 64 bits, which is 8 bytes.

Variable	Register
f	\$s0
g	\$s1
h	\$s2
a	\$s3
b	\$s4
temporary 1	\$t0
temporary 2	\$t1

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

In the hardware, the register file reads the source registers, the ALU computes, and the result is written back to a register.

Arrays Cannot Fit Inside One Register

An array usually lives in memory, not inside one register.

```
int A[10];
```

If one integer is 4 bytes, then 10 integers need 40 bytes.

A 32-bit register cannot store the whole 40-byte array.

But a register can store the starting address of the array.

```
$s3 = address of A[0]
```

To access A[8], the CPU uses:

```
address of A[8] = address of A[0] + 8 * 4 bytes
```

Since $8 * 4 = 32$, MIPS can write:

```
lw $t0, 32($s3)
```

This means: go to memory address $\$s3 + 32$, load one word, and put it into \$t0.

Load and Store

MIPS is a load-store architecture.

Arithmetic instructions usually work on registers, not memory directly.

Pattern:

```
memory
  ↓ lw
register
  ↓
ALU computes
  ↓
register
  ↓ sw
memory
g = h + A[8]
```

MIPS:

```
lw $t0, 32($s3)    # $t0 = A[8]
add $s1, $s2, $t0   # g = h + A[8]
A[10] = h + A[8];
```

MIPS:

```
lw $t0, 32($s3)    # $t0 = A[8]
add $t0, $s2, $t0   # $t0 = h + A[8]
sw $t0, 40($s3)     # A[10] = $t0
```

A[10] uses 40(\$s3) because $10 * 4 = 40$ bytes.

Immediate Operands

Some instructions can store a small constant directly inside the instruction.

This constant is called an immediate.

Field	op	rs	rt	immediate
bits	6	5	5	16

The immediate field is 16 bits, which is 2 bytes.

This does not mean the register is 2 bytes. In 32-bit MIPS, the register can still hold 32 bits.

2.4 Signed and Unsigned Numbers

A bit pattern itself has no meaning.

The interpretation rule decides what the bits mean.

Unsigned numbers have no negative values.

```
32-bit unsigned range:
0 to 4,294,967,295
```

For signed integers, MIPS commonly uses two's complement.

Two's Complement

If the most significant bit is 1, a signed two's complement number is negative.

To find its magnitude: invert all bits, then add 1.

```
1111 1111
invert -> 0000 0000
```

```
add 1 -> 0000 0001
```

So 1111 1111 represents -1 as signed two's complement.

The same bit pattern represents 255 if interpreted as unsigned.

2.5 Representing Instructions in the Computer

The computer does not truly understand assembly text.

It understands 0s and 1s.

So a MIPS assembly instruction must be represented as a 32-bit machine instruction.

Register Names Become Register Numbers

MIPS has 32 general-purpose registers, numbered 0 to 31.

A 5-bit field can represent 32 choices because $2^5 = 32$.

Register group	Numbers
\$zero	0
\$t0 - \$t7	8 - 15
\$s0 - \$s7	16 - 23
\$ra	31

```
add $t0, $s1, $s2
```

We think:

```
$t0 = $s1 + $s2
```

But the CPU sees:

```
register 8 = register 17 + register 18
```

R-format

R-format is used by many register-register operations.

```
add
sub
and
or
nor
sll
srl
```

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Example:

```
add $t0, $s1, $s2
```

Fields:

op	rs	rt	rd	shamt	funct
0	17	18	8	0	32

Binary:

```
000000 10001 10010 01000 00000 100000
```

op = 0 tells the CPU this is R-format.

funct = 32 tells the CPU the specific operation is add.

Metaphor: op is the department, funct is the worker inside that department.

I-format

I-format is used when an instruction needs an immediate, an offset, or a branch target offset.

```
addi
lw
sw
beq
bne
slti
andi
ori
```

op	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

```
lw $t0, 12($s3)
```

Fields:

op	rs	rt	immediate
lw	\$s3 base register	\$t0 target register	12 offset

Meaning:

```
$t0 = Memory[$s3 + 12]
```

J-format - Completed Note

J-format is used by jump-type instructions.

```
j Exit
```

J-format:

op	address
6 bits	26 bits

The instruction is still 32 bits total.

Unlike R-format, J-format does not need rs, rt, rd, shamt, or funct.

Jump does not add two registers or load from memory. It mainly tells the CPU: go to another instruction address.

Which jump-related instruction uses which format?

Instruction	Format	Meaning
j Label	J-format	unconditional jump to a label
jal Label	J-format	jump and link; used for procedure calls
beq rs, rt, Label	I-format	branch if equal
bne rs, rt, Label	I-format	branch if not equal
jr \$ra	R-format	jump to the address stored in a register

Important distinction:

```
j Exit          # J-format
beq $s1,$s2,L    # I-format
jr $ra          # R-format
```

So not every instruction that changes the program route is J-format.

beq and bne are conditional branches, so they use I-format.

j and jal are jump instructions, so they use J-format.

jr jumps to an address inside a register, so it uses R-format.

2.6 Logical Operations

2.6 teaches how the CPU directly operates on bits.

Arithmetic treats bits as numbers. Logical operations treat the value as a bit pattern.

and

AND returns 1 only when both bits are 1.

```
      1001
AND   1111
-----
      1001
and $t0, $s1, $s2
```

or

OR returns 1 if at least one bit is 1.

```
      1001
OR    1111
-----
      1111
or $t0, $s1, $s2
```

nor

NOR means NOT OR. First do OR, then invert the result.

```
      1001
OR    1010
-----
      1011
NOR result = 0100
```

Useful trick: any value NOR 0 becomes NOT of that value.

```
      1001
NOR   0000
-----
      0110
nor $t0, $s1, $zero    # $t0 = NOT $s1
```

sll and srl

sll means shift left logical.

```
sll $t0, $s1, 4
```

Shift the bit pattern in \$s1 left by 4 bits and store the result in \$t0.

```
0000 1001 << 4 = 1001 0000
```

srl means shift right logical.

```
srl $t0, $s1, 4
```

Shift the bit pattern in \$s1 right by 4 bits and fill the left side with 0s.

```
1001 1001 >> 4 = 0000 1001
```

Shift instructions make the shamt field meaningful.

Masks

AND and OR are often used with masks.

```
    1010 1101
AND 0000 1111
-----
    0000 1101
```

The 1s in the mask keep bits. The 0s clear bits.

```
    1010 0000
OR  0000 1111
-----
    1010 1111
```

2.7 Instructions for Making Decisions

2.7 teaches how the CPU makes decisions.

MIPS has no real if, else, while, or for instruction.

It builds those structures using labels, conditional branches, and jumps.

beq and bne

```
beq register1, register2, Label
```

If register1 equals register2, jump to Label.

```
bne register1, register2, Label
```

If register1 does not equal register2, jump to Label.

Labels such as Else:, Loop:, and Exit: are not instructions. They are road signs for control flow.

if-else Example

C code:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

Variable	Register
f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

A clean MIPS translation uses the opposite condition to jump to Else:

```
bne $s3, $s4, Else    # if i != j, go to Else
add $s0, $s1, $s2     # f = g + h
j    Exit             # skip else
```

```
Else:
sub $s0, $s1, $s2     # f = g - h
```

```
Exit:
```

MIPS has no else instruction. Else: is only a label.

Original diagram from the notes: if-else control flow

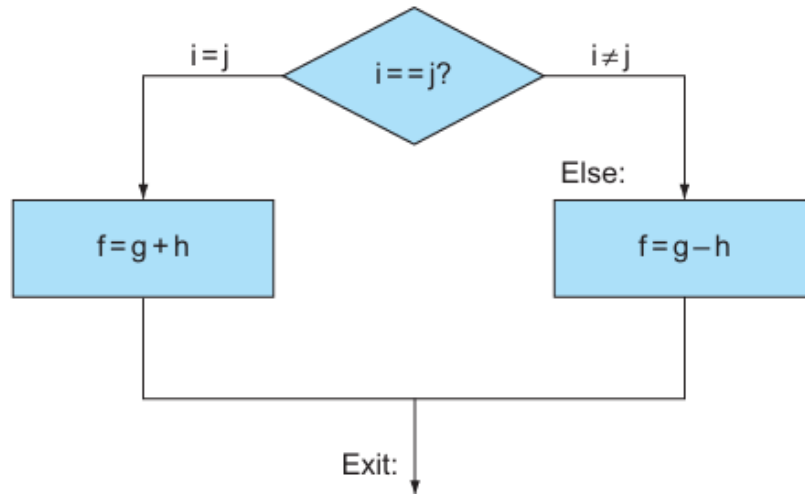


Figure: if $i == j$, execute $f = g + h$; otherwise execute $f = g - h$; both paths meet at Exit.

while Loop Example

C code:

```
while (save[i] == k)
    i += 1;
```

Meaning	Register
i	\$s3
k	\$s5
save base address	\$s6
save[i] value	\$t0
offset/address	\$t1

Loop:

```
sll $t1, $s3, 2      # $t1 = i * 4
add $t1, $t1, $s6    # $t1 = address of save[i]
lw  $t0, 0($t1)      # $t0 = save[i]
bne $t0, $s5, Exit   # if save[i] != k, exit
addi $s3, $s3, 1     # i = i + 1
j    Loop             # repeat
```

Exit:

The first three instructions are not the loop body. They are needed to get `save[i]` from memory before comparison.

Round 1:


```

i = 0
offset = 0 * 4 = 0
address = save base + 0
load save[0]
compare save[0] with k

```

```

Round 2:
i = 1
offset = 1 * 4 = 4
address = save base + 4
load save[1]
compare save[1] with k

```

\$s3 keeps i. \$s6 keeps the original base address. \$t1 temporarily becomes the address of save[i]. \$t0 keeps the value of save[i].

slt and slti

beq and bne can test equality and inequality. For less-than tests, use slt or slti.

```

slt $t0, $s1, $s2

```

If \$s1 < \$s2, then \$t0 = 1; otherwise, \$t0 = 0.

```

slti $t0, $s3, 10

```

If \$s3 < 10, then \$t0 = 1; otherwise, \$t0 = 0.

slt and slti do not jump by themselves. They create a 1 or 0 result. Then beq or bne uses that result to branch.

if (a < b) Example

```

if (a < b) {
    f = g + h;
}

```

Variable	Register
a	\$s1
b	\$s2
f	\$s3
g	\$s4
h	\$s5

```

slt $t0, $s1, $s2      # $t0 = 1 if a < b
beq $t0, $zero, Exit   # if a < b is false, skip
add $s3, $s4, $s5      # f = g + h

```

```

Exit:

```

Complete Practice Example

C code:

```
int A[] = {2, 7, 3, 9};
int i = 0;

while (i < 4) {
    if (A[i] < 5) {
        A[i] = 0;
    }
    i = i + 1;
}
```

Correct MIPS version:

```
.data
A: .word 2, 7, 3, 9

.text
main:
    addi $s3, $zero, 0      # i = 0
    la   $s6, A             # $s6 = A base address

Loop:
    slti $t0, $s3, 4        # $t0 = 1 if i < 4
    beq  $t0, $zero, Exit   # if i >= 4, exit

    sll  $t1, $s3, 2        # $t1 = i * 4
    add  $t1, $t1, $s6      # $t1 = address of A[i]
    lw   $t2, 0($t1)        # $t2 = A[i]

    slti $t3, $t2, 5        # $t3 = 1 if A[i] < 5
    beq  $t3, $zero, Skip   # if A[i] >= 5, skip

    add  $t2, $zero, $zero  # $t2 = 0
    sw   $t2, 0($t1)        # A[i] = 0

Skip:
    addi $s3, $s3, 1        # i = i + 1
    j    Loop

Exit:
```

Register	Role
\$s3	i
\$s6	A base address
\$t0	result of i < 4
\$t1	A[i] address
\$t2	A[i] value
\$t3	result of A[i] < 5

Important rule: do not overwrite \$s3 with a comparison result, because \$s3 is i.

Quick Final Summary

Concept	Main idea
R-format	register-register operations; op, rs, rt, rd, shamt, funct
I-format	immediate, offset, load/store, and conditional branch instructions
J-format	unconditional jump instructions such as j and jal
beq/bne	conditional branches
j	unconditional jump
slt/slti	make a less-than comparison result: 1 or 0
array access	base address + i * 4, then lw or sw

The real goal is not to memorize MIPS as a language.

The real goal is to understand how high-level code becomes simple CPU instructions.